Snowflake Iceberg Tables with S3 Integration - Complete Cheatsheet

Part 1: Set Up AWS Resources

First, we'll create the necessary AWS resources that Snowflake will use to store Iceberg table data.

Let us set some environment variables for convinience and use:

```
# Set bucket name in environment variable
export S3_BUCKET_NAME="kamesh-snowflake-iceberg-demo"
export AWS_ROLE_NAME="snowflake-iceberg-role"
export AWS_REGION="us-west-2"
```

> NOTE: If `S3_BUCKET_NAME` already exist try using a different name

1. S3 Bucket Creation

Create an S3 bucket in the same region `us-west-2($AWS_REGION)` as your Snowflake warehouse:

```
aws s3api create-bucket \
    --bucket $S3_BUCKET_NAME \
    --region $AWS_REGION \
    --create-bucket-configuration LocationConstraint=$AWS_REGION
```

Enable versioning (recommended for data consistency)

```
aws s3api put-bucket-versioning \
    --bucket $S3_BUCKET_NAME \
    --versioning-configuration Status=Enabled
```

2. IAM Role Creation

Create an IAM role that Snowflake will assume to access the S3 bucket:

```
# Create IAM role with basic trust policy
aws iam create-role \
```

```
    --role-name $AWS_ROLE_NAME \
    --assume-role-policy-document
        '{"Version":"2012-10-17","Statement":
        [{"Effect":"Allow","Principal":
        {"Service":"s3.amazonaws.com"},"Action":"sts:AssumeRole"}]}'
```

Store the role ARN for later use

```
export ROLE_ARN=$(aws iam get-role --role-name $AWS_ROLE_NAME --
        query 'Role.Arn' --output text)
echo "Role ARN: $ROLE_ARN"
```

3. S3 Access Policy Creation

Create and attach an IAM policy that grants the required S3 permissions:

```
# Create the policy definition file (iceberg-s3-policy.json)
cat > iceberg-s3-policy.json << EOF
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "s3:PutObject",
                "s3:GetObject",
                "s3:GetObjectVersion",
                "s3:DeleteObject",
                "s3:DeleteObjectVersion"
            ],
            "Resource": "arn:aws:s3:::${S3_BUCKET_NAME}/*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": "arn:aws:s3:::${S3_BUCKET_NAME}",
            "Condition": {
                "StringLike": {
                    "s3:prefix": [
                        "*"
                    ]
                }
            }
        }
    ]
}
EOF
```

Create IAM policy

```
aws iam create-policy \
    --policy-name snowflake-iceberg-policy \
    --policy-document file://iceberg-s3-policy.json
```

Attach policy to role

```
aws iam attach-role-policy \
    --role-name $AWS_ROLE_NAME \
    --policy-arn arn:aws:iam::$(aws sts get-caller-identity --
query 'Account' --output text):policy/snowflake-iceberg-policy
```

Part 3: Update AWS Trust Relationship

After creating the external volume in Snowflake, you need to update the AWS IAM role with the trust relationship using values generated by Snowflake.

1. Extract Snowflake Integration Values

Once you have created the external volume in Snowflake, extract the necessary values:

```
export SNOWFLAKE_IAM_USER_ARN=$(snow sql --query "DESC EXTERNAL
        VOLUME ICEBERG_DEMO_VOLUME" --format json | jq -r '.
        [1].property_value|fromjson.STORAGE_AWS_IAM_USER_ARN')
echo "IAM User ARN: $SNOWFLAKE_IAM_USER_ARN"
```

Get the External ID (automatically generated by Snowflake) and set as environment variable

```
export SNOWFLAKE_EXTERNAL_ID=$(snow sql --query "DESC EXTERNAL
        VOLUME ICEBERG_DEMO_VOLUME" --format json | jq -r '.
        [1].property_value|fromjson.STORAGE_AWS_EXTERNAL_ID')
echo "External ID: $SNOWFLAKE_EXTERNAL_ID"
```

2. Update IAM Role Trust Relationship

View the current trust policy:

```
echo "Current trust policy before update:"
aws iam get-role --role-name $AWS_ROLE_NAME --query
'Role.AssumeRolePolicyDocument' --output json
```

Update and apply the trust policy:

```
cat > snowflake-trust-policy.json << EOF
{
  "Version": "2012-10-17",
```

```
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "${SNOWFLAKE_IAM_USER_ARN}"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "${SNOWFLAKE_EXTERNAL_ID}"
        }
      }
    }
  ]
}
EOF
```

Update trust relationship on the role:

```
aws iam update-assume-role-policy \
    --role-name $AWS_ROLE_NAME \
    --policy-document file://snowflake-trust-policy.json
```

> NOTE: Give it few seconds for the trust policy to refresh in AWS.

Run the following command to view the updated Trust policy. It should have the `$SNOWFLAKE_IAM_USER_ARN` and `$SNOWFLAKE_EXTERNAL_ID` updated in the policy:

```
echo "Updated trust policy:"
aws iam get-role --role-name $AWS_ROLE_NAME --query
'Role.AssumeRolePolicyDocument' --output json
```

Part 3: Set Up Snowflake Objects

Now that we have the AWS resources ready, we can configure Snowflake to use them using the `snow sql --stdin` approach.

```
export SNOWLFLAKE_DEAULT_CONNECTION_NAME="<your connection name
e.g. trial>"
```

1. Setting Up Snowflake Database and Schema

```
# Create initial Snowflake database and schema
snow sql --stdin << EOF
USE ROLE ACCOUNTADMIN;

CREATE DATABASE IF NOT EXISTS ICEBERG_DEMO_DB;
USE DATABASE ICEBERG_DEMO_DB;
```

```
CREATE SCHEMA IF NOT EXISTS ICEBERG_SCHEMA;
USE SCHEMA ICEBERG_SCHEMA;
EOF
```

2. Setting Up External Volume with S3

Create the external volume using the AWS resources:

```
# Create external volume in Snowflake
snow sql --stdin << EOF
USE ROLE ACCOUNTADMIN;
USE DATABASE ICEBERG_DEMO_DB;
USE SCHEMA ICEBERG_SCHEMA;

CREATE OR REPLACE EXTERNAL VOLUME ICEBERG_DEMO_VOLUME
  STORAGE_LOCATIONS =
      (
        (
            NAME = 'iceberg-demo-location'
            STORAGE_PROVIDER = 'S3'
            STORAGE_BASE_URL = 's3://$S3_BUCKET_NAME/'
            STORAGE_AWS_ROLE_ARN = '$ROLE_ARN'
        )
      )
  ALLOW_WRITES = TRUE;
EOF
```

3. Creating Iceberg Table

```
# Create Iceberg table
snow sql --stdin << EOF
USE ROLE ACCOUNTADMIN;
USE DATABASE ICEBERG_DEMO_DB;
USE SCHEMA ICEBERG_SCHEMA;

CREATE ICEBERG TABLE IF NOT EXISTS TODO_ITEMS (
 id VARCHAR,
 task VARCHAR,
 status VARCHAR,
 created_at TIMESTAMP_NTZ,
 updated_at TIMESTAMP_NTZ
)
  CATALOG = 'SNOWFLAKE'
  EXTERNAL_VOLUME = 'ICEBERG_DEMO_VOLUME'
  BASE_LOCATION = 'todo_app_data';
EOF
```

Enable schema evolution to allow future field additions

```
snow sql --stdin << EOF
USE ROLE ACCOUNTADMIN;
```

```
USE DATABASE ICEBERG_DEMO_DB;
USE SCHEMA ICEBERG_SCHEMA;
ALTER ICEBERG TABLE TODO_ITEMS set ENABLE_SCHEMA_EVOLUTION = true;
EOF
```

4. Verify S3 Integration

Insert a sample record into your Iceberg table and verify that the data and metadata are properly stored in the S3 bucket:

```
# Insert a test record
snow sql --stdin << EOF
USE ROLE ACCOUNTADMIN;
USE DATABASE ICEBERG_DEMO_DB;
USE SCHEMA ICEBERG_SCHEMA;

INSERT INTO TODO_ITEMS (id, task, status, created_at, updated_at)
VALUES ('test', 'Integration test', 'PENDING',
        CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP());
EOF
```

Check for files in the S3 bucket

```
aws s3 ls s3://$S3_BUCKET_NAME/ --recursive
```

The S3 bucket structure should look similar to:

```
2025-04-01 12:23:26       1024 todo_app_data.a43VCEFf/data/32/
snow_V8coiQtGf2o_APg1YUMfMhg_0_1_002.parquet
2025-04-01 12:23:11       1223 todo_app_data.a43VCEFf/metadata/
00000-************************************.metadata.json
2025-04-01 12:23:28       1954 todo_app_data.a43VCEFf/metadata/
00001-************************************.metadata.json
2025-04-01 12:23:28       6909 todo_app_data.a43VCEFf/metadata/
1743490405955000000-9pkvny8A5NECsUIiJwmK1A.avro
2025-04-01 12:23:28       4224 todo_app_data.a43VCEFf/metadata/
snap-1743490405955000000-************************************.avro
```

Part 4: Using Iceberg Tables

Now that the integration is set up, you can start using your Iceberg tables.

Query and Manipulate Data

```
# Data manipulation examples
snow sql --stdin << EOF
USE ROLE ACCOUNTADMIN;
USE DATABASE ICEBERG_DEMO_DB;
USE SCHEMA ICEBERG_SCHEMA;
```

```
-- Insert todo items
INSERT INTO TODO_ITEMS (id, task, status, created_at, updated_at)
VALUES
  ('1', 'Complete Iceberg integration', 'PENDING',
        CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP()),
  ('2', 'Test time travel functionality', 'PENDING',
        CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP()),
  ('3', 'Document schema evolution', 'IN_PROGRESS',
        CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP());

-- Query all todo items
SELECT * FROM TODO_ITEMS;

-- Query only pending tasks
SELECT * FROM TODO_ITEMS WHERE status = 'PENDING';

-- Update task status
UPDATE TODO_ITEMS
SET status = 'COMPLETED', updated_at = CURRENT_TIMESTAMP()
WHERE id = '1';

-- Delete completed tasks
DELETE FROM TODO_ITEMS WHERE status = 'COMPLETED';
EOF
```

Schema Evolution

```
# Schema evolution examples
snow sql --stdin << EOF
USE ROLE ACCOUNTADMIN;
USE DATABASE ICEBERG_DEMO_DB;
USE SCHEMA ICEBERG_SCHEMA;

-- Add priority column to todo items
ALTER ICEBERG TABLE TODO_ITEMS ADD COLUMN RECORD_METADATA
        OBJECT();

-- Add priority column to todo items
ALTER ICEBERG TABLE TODO_ITEMS ADD COLUMN priority VARCHAR;

-- Add due_date column
ALTER ICEBERG TABLE TODO_ITEMS ADD COLUMN due_date DATE;

-- Rename status column to task_status for clarity
ALTER ICEBERG TABLE TODO_ITEMS RENAME COLUMN status TO
        task_status;

-- Drop unused column if needed
ALTER ICEBERG TABLE TODO_ITEMS DROP COLUMN RECORD_METADATA;

-- Change task column to allow longer descriptions
```

```
ALTER ICEBERG TABLE TODO_ITEMS ALTER COLUMN task SET DATA TYPE
        VARCHAR(500);
EOF
```

Cleanup Resources

When you're done with your testing or if you need to remove the resources, you can use the following commands to clean up.

1. Cleaning Up Snowflake Resources

```
snow sql --stdin << EOF
USE ROLE ACCOUNTADMIN;

-- Drop the Iceberg table first
DROP ICEBERG TABLE IF EXISTS
        ICEBERG_DEMO_DB.ICEBERG_SCHEMA.TODO_ITEMS;

-- Drop the external volume
DROP EXTERNAL VOLUME IF EXISTS
        ICEBERG_DEMO_DB.ICEBERG_SCHEMA.ICEBERG_DEMO_VOLUME;

-- Drop the schema and database
DROP SCHEMA IF EXISTS ICEBERG_DEMO_DB.ICEBERG_SCHEMA;
DROP DATABASE IF EXISTS ICEBERG_DEMO_DB;
EOF
```

2. Cleaning Up AWS Resources

First, remove all objects and their versions

```
aws s3api list-object-versions --bucket $S3_BUCKET_NAME --output
        json --query '{Objects: Versions[].
        {Key:Key,VersionId:VersionId}}' | \
jq 'if .Objects != null then {Objects: .Objects} else {} end' | \
aws s3api delete-objects --bucket $S3_BUCKET_NAME --delete
        file:///dev/stdin
```

Then delete all delete markers

```
aws s3api list-object-versions --bucket $S3_BUCKET_NAME --output
        json --query '{Objects: DeleteMarkers[].
        {Key:Key,VersionId:VersionId}}' | \
jq 'if .Objects != null then {Objects: .Objects} else {} end' | \
aws s3api delete-objects --bucket $S3_BUCKET_NAME --delete
        file:///dev/stdin
```

Finally delete the bucket

```
aws s3api delete-bucket --bucket $S3_BUCKET_NAME
```

Detach policy from role

```
POLICY_ARN=$(aws iam list-attached-role-policies --role-name
        $AWS_ROLE_NAME --query "AttachedPolicies[?
        PolicyName=='snowflake-iceberg-policy'].PolicyArn" --
        output text)
aws iam detach-role-policy --role-name $AWS_ROLE_NAME --policy-
        arn $POLICY_ARN
```

Delete the IAM role

```
aws iam delete-role --role-name $AWS_ROLE_NAME
```

Delete the policy

```
aws iam delete-policy --policy-arn $POLICY_ARN
```

Remove local files created during setup

```
rm -f iceberg-s3-policy.json snowflake-trust-policy.json
```

3. Verification

Verify S3 bucket is deleted

```
# If no output, the bucket has been deleted
aws s3 ls | grep $S3_BUCKET_NAME
```

Verify IAM role is deleted

```
# Should return an error indicating the role doesn't exist
aws iam get-role --role-name $AWS_ROLE_NAME 2>&1 | grep
        "NoSuchEntity"
```

Verify Snowflake resources are gone

```
# Should return no results
snow sql --query "SHOW DATABASES LIKE 'ICEBERG_DEMO_DB'"
```